Python Regular Expressions (RegEx)

1. Introduction

A **Regular Expression (RegEx)** is a sequence of characters that defines a search pattern. It is commonly used for string searching, text matching, and manipulation in Python.

In Python, the built-in `re` **module** provides functions to work with RegEx patterns effectively.

---

2. Example of a RegEx Pattern

```
regex
    ^a...s$
```

Explanation of the Pattern `^a...s$`:

- `^` → Asserts that the string must **start with** `a`
- `...` → Matches **any three characters** (letters, digits, or symbols)
- `s$` → Asserts that the string must **end with** `s`

This pattern matches **any 5-letter string starting with** `a` **and ending with** `s`.

3. Example Match

Pattern	String	Matched?
`^a...s$`	`abs`	❌ No match (only 3 letters)
`^a...s$`	`alias`	✅ Match
`^a...s$`	`abyss`	✅ Match
`^a...s$`	`Alias`	❌ No match (case-sensitive)
`^a...s$`	`An abacus`	❌ No match (not 5 letters)

4. Using RegEx in Python

Python's `re` **module** provides powerful tools for working with regular expressions. Below is a simple example demonstrating how to use the `re` module to match the pattern `^a...s$`:

In [1]:
```python
import re

# Define the RegEx pattern
pattern = r'^a...s$'

# Test strings
test_strings = ['alias', 'abyss', 'abs', 'Alias', 'An abacus']

# Check each string against the pattern
for s in test_strings:
    if re.match(pattern, s):
        print(f"'{s}' → Match")
    else:
        print(f"'{s}' → No match")
```

```
'alias' → Match
'abyss' → Match
'abs' → No match
'Alias' → No match
'An abacus' → No match
```

Explanation of `re.match()`

The `re.match()` function is used to search for a pattern **at the beginning of a string.**

- If the search is successful, it returns a **match object.**
- If the search fails, it returns **None.**

In the example above, `re.match()` checks whether each test string conforms to the pattern `^a...s$` at the start of the string.

Specify Pattern Using RegEx in Python

To define patterns in **Regular Expressions (RegEx)**, we use **metacharacters.**
These are special characters that carry unique meanings and help us build flexible matching rules.

For example:

- `^` → matches the **start** of a string
- `$` → matches the **end** of a string

```python
import re

# Example: match strings that start with 'a' and end with 'z'
pattern = r"^a.*z$"
text1 = "abcz"
text2 = "zebra"
```

```
print(bool(re.match(pattern, text1)))  #  True
print(bool(re.match(pattern, text2)))  #  False
```

MetaCharacters in RegEx

Metacharacters are interpreted specially by the RegEx engine (not treated as normal text). They help define flexible and powerful search patterns.

Complete List of MetaCharacters

`[] . ^ $ * + ? {} () \ |`

Quick Meaning of Common MetaCharacters in RegEx

- `[]` → Matches **any single character** inside the brackets
- `.` → Matches **any character** (except newline by default)
- `^` → Matches the **beginning** of a string
- `$` → Matches the **end** of a string
- `*` → Matches **0 or more repetitions**
- `+` → Matches **1 or more repetitions**
- `?` → Matches **0 or 1 occurrence** (optional)
- `{n}` → Matches **exactly n times**
- `{n,m}` → Matches between **n and m times**
- `()` → Groups expressions
- `\` → Escapes a special character
- `|` → Acts like **OR** (alternative matching)

In [2]:
```
# 1. [] → character set
print(re.findall(r"[aeiou]", "python regex"))
# Output: ['o', 'e', 'e']  (all vowels matched)
```

```
['o', 'e', 'e']
```

In [3]:
```
# 2. . → any character
print(re.findall(r".at", "cat bat hat rat mat 123at"))
# Output: ['cat', 'bat', 'hat', 'rat', 'mat', '123at']
```

```
['cat', 'bat', 'hat', 'rat', 'mat', '3at']
```

In [4]:
```
# 3. * → zero or more
print(re.findall(r"ab*", "a ab abb abbb xyz"))
# Output: ['a', 'ab', 'abb', 'abbb']
```

```
['a', 'ab', 'abb', 'abbb']
```

In [5]:
```
# 4. + → one or more
print(re.findall(r"ab+", "a ab abb abbb xyz"))
# Output: ['ab', 'abb', 'abbb']
```

```
['ab', 'abb', 'abbb']
```

In [6]:
```python
# 5. ? → zero or one
print(re.findall(r"colou?r", "color colour"))
# Output: ['color', 'colour']
```

```
['color', 'colour']
```

In [7]:
```python
# 6. {n,m} → repetition range
print(re.findall(r"\d{2,4}", "My numbers are 12, 123, 1234, 12345"))
# Output: ['12', '123', '1234', '1234']
```

```
['12', '123', '1234', '1234']
```

1. [ ] - Square Brackets

Square brackets specify a **set of characters** you want to match.

Expression	String	Matched?
`[abc]`	`a`	1 match
`[abc]`	`ac`	2 matches
`[abc]`	`Hey Jude`	No match
`[abc]`	`abc de ca`	5 matches

Here, `[abc]` will match if the string contains **any of** `a`, `b`, **or** `c`.

Ranges inside Square Brackets

You can specify a range of characters using `-`:

- `[a-c]` → matches `a` or `b` or `c`
- `[a-z]` → matches any lowercase letter
- `[A-Z]` → matches any uppercase letter
- `[0-3]` → matches `0`, `1`, `2`, or `3`
- `[0-9]` → matches any digit
- `[A-Za-z0-9]` → matches any single alphanumeric character

Negation with `^`

Placing `^` at the beginning inside brackets inverts the set:

- `[^abc]` → matches any character **except** `a`, `b`, or `c`
- `[^0-9]` → matches any non-digit character

Using `re.findall()` in Python

The `re.findall()` function **searches the string for all occurrences** that match a pattern and **returns them as a list.**

Syntax:

```
re.findall(pattern, string, flags=0)
```

- `pattern` → the regex pattern to search
- `string` → the text in which to search
- `flags` → optional modifiers (like `re.IGNORECASE`)

In [8]:
```
import re

# Match vowels
pattern = r"[aeiou]"
text = "regular expressions"
print(re.findall(pattern, text))
# Output: ['e', 'u', 'a', 'e', 'o', 'i', 'o']
```

```
['e', 'u', 'a', 'e', 'e', 'i', 'o']
```

In [9]:
```
# Match digits
pattern = r"[0-9]"
text = "Year 2025 has digits"
print(re.findall(pattern, text))
# Output: ['2', '0', '2', '5']
```

```
['2', '0', '2', '5']
```

In [10]:
```
# Match non-digits
pattern = r"[^0-9]"
text = "A1B2C3"
print(re.findall(pattern, text))
# Output: ['A', 'B', 'C']
```

```
['A', 'B', 'C']
```

2. `.` - Dot

The dot `.` matches **any single character** except a newline (`'\n'`).

Expression	String	Matched?
`.`	`a`	1 match
`.`	`ac`	2 matches (each character individually)
`..`	`acd`	1 match
`..`	`acde`	2 matches (matches two characters at a time)

Using `re.findall()` in Python

The `re.findall()` function **searches the string for all occurrences** that match a pattern and **returns them as a list.**

Syntax:

```
re.findall(pattern, string, flags=0)
```

- `pattern` → the regex pattern to search
- `string` → the text in which to search
- `flags` → optional modifiers (like `re.IGNORECASE`)

In [11]:
```
import re

# Match any character followed by 'c'
pattern = r".c"
text = "abc adc aec"
print(re.findall(pattern, text))
# Output: ['bc', 'dc', 'ec']
```

```
['bc', 'dc', 'ec']
```

In [12]:
```
# Match two characters together
pattern = r".."
text = "abcd"
print(re.findall(pattern, text))
# Output: ['ab', 'cd']
```

```
['ab', 'cd']
```

In [13]:
```
# Match all single characters
pattern = r"."
text = "abc"
print(re.findall(pattern, text))
# Output: ['a', 'b', 'c']
```

```
['a', 'b', 'c']
```

Explanation:

- A single `.` matches **any one character.**
- `..` matches **any two consecutive characters.**
- `re.findall()` returns all **non-overlapping matches** as a list.

3. `^` - Caret

The caret `^` is used to check if a string **starts with** a certain character or pattern.

- `r'^substring'` → matches strings that **start with** `substring`
  Example: `r'^love'` matches any string starting with `"love"`
- `r'[^abc]'` → matches any character **except** `a`, `b`, or `c` (negation inside square brackets)

Expression	String	Matched?
`^a`	`a`	1 match
`^a`	`abc`	1 match
`^a`	`bac`	No match
`^a.`	`abc`	1 match
`^a.`	`acd`	1 match

Using `re.findall()` in Python

In [14]:
```python
import re

# Text to search
text = "abc acd bac"

# Split the string into words
words = text.split()

# Pattern to match words starting with 'a'
pattern = r"^a"

# Check each word
matches = [re.findall(pattern, word) for word in words if re.findall(pattern, word)]

print(matches)
# Output: [['a'], ['a']]  (matches 'abc' and 'acd')
```

```
[['a'], ['a']]
```

In [15]:
```python
# Match any character at start followed by 'b'
pattern = r"^ab"
text = "abc abd axy"
print(re.findall(pattern, text))
# Output: ['ab']  (matches 'ab' at the start of 'abc')
```

```
['ab']
```

Explanation:

- `^` at the beginning of a pattern checks the **start of the string.**
- Inside square brackets `[ ]`, `^` **negates the set**, matching any character **not in the set**.
- `re.findall()` returns all **non-overlapping matches** as a list.

4. `$` - Dollar

The dollar symbol `$` is used to check if a string **ends with** a certain character or pattern.

Expression	String	Matched?
`a$`	`a`	1 match
`a$`	`formula`	1 match
`a$`	`cab`	No match

Using `re.findall()` in Python

In [16]:
```python
import re

# Text to search
text = "a formula cab"

# Split the string into words to check each individually
words = text.split()

# Pattern to match words ending with 'a'
pattern = r"a$"

# Check each word
matches = [re.findall(pattern, word) for word in words if re.findall(pattern, word)]

print(matches)
# Output: [['a'], ['a']]  (matches 'a' and 'formula')
```

```
[['a'], ['a']]
```

In [17]:
```python
import re

# Text to search
text = "cab tab abc"

# Split the string into words to check each individually
words = text.split()

# Pattern to match words ending with 'b'
```

```
pattern = r"b$"

# Check each word
matches = [re.findall(pattern, word) for word in words if re.findall(pattern, word)

print(matches)
# Output: [['b'], ['b']]  (matches 'cab' and 'tab')
```

```
[['b'], ['b']]
```

5. `*` - Star

The star symbol `*` matches **zero or more occurrences** of the pattern to its left.

Expression	String	Matched?
`ma*n`	**`mn`**	1 match (zero `a`)
`ma*n`	**`man`**	1 match (one `a`)
`ma*n`	**`maaan`**	1 match (three `a`s)
`ma*n`	**`main`**	No match (`a` not followed by `n`)
`ma*n`	**`woman`**	1 match (`an` at the end)

Using `re.findall()`

In Python, we use **`re.findall(pattern, string)`** to return all **non-overlapping matches** of a pattern as a list.

In [18]:
```
import re

pattern = r"ma*n"
text = "mn man maaan main woman"
print(re.findall(pattern, text))
# Output: ['mn', 'man', 'maaan', 'man']
```

```
['mn', 'man', 'maaan', 'man']
```

Explanation:

- `*` allows **zero, one, or many** occurrences of the preceding character.
- `mn` matches because zero `a`s are allowed.
- `man` matches because one `a` is allowed.
- `maaan` matches because multiple `a`s are allowed.
- `main` does not match because after `a` comes `i` instead of `n`.
- `woman` matches because it ends with `an`.

6. `+` - Plus

The plus symbol `+` matches **one or more occurrences** of the pattern to its left.

Expression	String	Matched?
`ma+n`	`mn`	No match (requires at least one `a`)
`ma+n`	`man`	1 match (one `a`)
`ma+n`	`maaan`	1 match (three `a`s)
`ma+n`	`main`	No match (`a` not followed by `n`)
`ma+n`	`woman`	1 match (`an` at the end)

Using `re.findall()`

In [19]:
```python
import re

pattern = r"ma+n"
text = "mn man maaan main woman"
print(re.findall(pattern, text))
# Output: ['man', 'maaan', 'man']
```

```
['man', 'maaan', 'man']
```

Explanation:

- `+` requires **at least one** occurrence of the preceding character.
- `mn` does not match because no `a` is present.
- `man` matches because it has one `a`.
- `maaan` matches because it has multiple `a`s.
- `main` does not match because after `a` comes `i` instead of `n`.
- `woman` matches because it ends with `an`.

7. `?` - Question Mark

The question mark symbol `?` matches **zero or one occurrence** of the pattern to its left.

Expression	String	Matched?
`ma?n`	`mn`	1 match (zero `a`)
`ma?n`	`man`	1 match (one `a`)
`ma?n`	`maaan`	No match (more than one `a`)

Expression	String	Matched?
`ma?n`	`main`	No match (`a` not followed by `n`)
`ma?n`	`woman`	1 match (`an` at the end)

Using `re.findall()`

In [20]:
```
import re

pattern = r"ma?n"
text = "mn man maaan main woman"
print(re.findall(pattern, text))
# Output: ['mn', 'man', 'man']
```

```
['mn', 'man', 'man']
```

Explanation:

- `?` allows **zero or one** occurrence of the preceding character.
- `mn` matches because zero `a` s are allowed.
- `man` matches because one `a` is allowed.
- `maaan` does not match because it has more than one `a` .
- `main` does not match because after `a` comes `i` instead of `n` .
- `woman` matches because it ends with `an` .

8. `{}` - Braces

Braces `{}` are used to specify the **number of occurrences** of the pattern to the left.

- `{n}` → Exactly `n` times
- `{n,}` → At least `n` times
- `{n,m}` → Between `n` and `m` times

Example 1: Character repetitions

Expression	String	Matched?
`a{2,3}`	`abc dat`	No match
`a{2,3}`	`abc daat`	1 match (`aa` in `daat`)
`a{2,3}`	`aabc daaat`	2 matches (`aa` in `aabc` , `aaa` in `daaat`)
`a{2,3}`	`aabc daaaat`	2 matches (`aa` in `aabc` , `aaa` in `daaaat`)

Example 2: Digit repetitions

Expression	String	Matched?
`[0-9]{2,4}`	**`ab123csde`**	1 match (`123`)
`[0-9]{2,4}`	**`12 and 345673`**	3 matches (`12` , `3456` , `73`)
`[0-9]{2,4}`	**`1 and 2`**	No match

Using `re.findall()`

In [21]:
```
import re

# Example 1: Character repetitions
pattern = r"a{2,3}"
text = "abc daat aabc daaat aabc daaaat"
print(re.findall(pattern, text))
# Output: ['aa', 'aa', 'aaa', 'aa', 'aaa']
```

```
['aa', 'aa', 'aaa', 'aa', 'aaa']
```

In [22]:
```
# Example 2: Digit repetitions
pattern = r"[0-9]{2,4}"
text = "12 and 345673 and ab123csde"
print(re.findall(pattern, text))
# Output: ['12', '3456', '73', '123']
```

```
['12', '3456', '73', '123']
```

Explanation:

- `{n}` specifies an **exact number** of repetitions.
- `{n,}` specifies **at least n** repetitions.
- `{n,m}` specifies a **range of repetitions** between n and m.
- `[0-9]{2,4}` matches numbers with **2 to 4 digits**.

9. `|` - Alternation

The vertical bar `|` is used for **alternation**, which works like a logical **OR** operator in regex. It matches **either the pattern on the left or the pattern on the right**.

Expression	String	Matched?	
`a	b`	**`cde`**	No match
`a	b`	**`ade`**	1 match (`a`)
`a	b`	**`acdbea`**	3 matches (`a` , `b` , `a`)

Using `re.findall()`

In [23]:
```python
import re

pattern = r"a|b"
text = "cde ade acdbea"
print(re.findall(pattern, text))
# Output: ['a', 'a', 'b', 'a']
```

```
['a', 'a', 'b', 'a']
```

Explanation:

- `|` matches **either the left or right pattern**.
- `a|b` matches any occurrence of `a` **or** `b` in the string.
- `re.findall()` returns all **non-overlapping matches** as a list.

10. `()` - Group

Parentheses `()` are used to **group sub-patterns** in regex. This allows applying quantifiers or alternation to the entire group.

For example, `(a|b|c)xz` matches any string where `a`, `b`, or `c` is **followed by** `xz`.

Expression	String	Matched?		
`(a	b	c)xz`	**ab xz**	No match
`(a	b	c)xz`	**abxz**	1 match (axz)
`(a	b	c)xz`	**axz cabxz**	2 matches (`axz` , `cxz`)

Using `re.findall()`

In [24]:
```python
import re

pattern = r"(a|b|c)xz"
text = "ab xz abxz axz cabxz"
print(re.findall(pattern, text))
# Output: ['a', 'a', 'a', 'c']
```

```
['b', 'a', 'b']
```

Explanation:

- `()` groups **sub-patterns** together.
- `(a|b|c)xz` matches any occurrence of `a`, `b`, or `c` **followed by** `xz`.
- `re.findall()` returns the part of the string matched by the group.

11. `\` - Backslash

The backslash `\` is used to **escape special characters** in regex, so they are treated as **literal characters** instead of metacharacters.

For example:

- `\$a` matches a string that contains `$` followed by `a`.
  - Here, `$` is **not interpreted** as "end of string" because it is escaped with `\`.

Usage tip:

- If you are unsure whether a character has a special meaning in regex, place a `\` in front of it to ensure it is treated **literally**.

Using `re.findall()`

In [25]:
```
import re

pattern = r"\$a"
text = "This costs $a5 and $a10"
print(re.findall(pattern, text))
# Output: ['$a', '$a']
```

```
['$a', '$a']
```

Explanation:

- The backslash `\` **escapes special characters**.
- `$` normally represents the **end of a string**, but `\$` matches the **literal** `$`.
- `re.findall()` returns all occurrences of the **escaped pattern** in the string.

Python Regular Expressions (RegEx) - Complete Notes

Special Sequences

Special sequences in regex simplify commonly used patterns. They are shortcuts for frequently used character classes or positions.

Here's a list of commonly used special sequences:

Sequence	Meaning
`\d`	Matches any **digit**; equivalent to `[0-9]`
`\D`	Matches any **non-digit**; equivalent to `[^0-9]`
`\w`	Matches any **alphanumeric character** (letters, digits, underscore); equivalent to `[a-zA-Z0-9_]`
`\W`	Matches any **non-alphanumeric character**; equivalent to `[^a-zA-Z0-9_]`
`\s`	Matches any **whitespace** character (space, tab, newline)
`\S`	Matches any **non-whitespace** character
`\b`	Matches a **word boundary**
`\B`	Matches **not a word boundary**

Special Sequences in Python Regex

`\A` - Matches if the specified characters are at the **start of a string.**

Expression	String	Matched?
`\Athe`	`the sun`	Match
`\Athe`	`In the sun`	No match

`\b` - Matches if the specified characters are at the **beginning or end of a word.**

Expression	String	Matched?
`\bfoo`	`football`	Match
`\bfoo`	`a football`	Match
`\bfoo`	`afootball`	No match
`foo\b`	`the foo`	Match
`foo\b`	`the afoo test`	Match
`foo\b`	`the afootest`	No match

`\B` - Opposite of `\b`. Matches if the specified characters are **not at the beginning or end of a word.**

Expression	String	Matched?
`\Bfoo`	`football`	No match
`\Bfoo`	`a football`	No match
`\Bfoo`	`afootball`	Match
`foo\B`	`the foo`	No match
`foo\B`	`the afoo test`	No match
`foo\B`	`the afootest`	Match

`\d` - Matches any **digit**, equivalent to `[0-9]`.

Expression	String	Matched?
`\d`	`Year 2025`	4 matches (`2` , `0` , `2` , `5`)

`\D` - Matches any **non-digit**, equivalent to `[^0-9]`.

Expression	String	Matched?
`\D`	`Year 2025`	Matches all letters and spaces

`\s` - Matches any **whitespace character** (space, tab, newline).

Expression	String	Matched?
`\s`	`Hello World`	1 match (space)

`\S` - Matches any **non-whitespace character.**

Expression	String	Matched?
`\S`	`Hello World`	Matches all letters

`\w` - Matches any **alphanumeric character** (letters, digits, underscore), equivalent to `[a-zA-Z0-9_]`.

Expression	String	Matched?
`\w`	`Hello_123!`	Matches `H e l l o _ 1 2 3`

`\W` - Matches any **non-alphanumeric character**, equivalent to `[^a-zA-Z0-9_]`.

Expression	String	Matched?
`\W`	`Hello_123!`	Matches `!`

`\Z` - Matches if the specified characters are at the **end of a string.**

Expression	String	Matched?
`Python\Z`	`I like Python`	Match
`Python\Z`	`I like Python Programming`	No match
`Python\Z`	`Python is fun.`	No match

Summary - MetaCharacters

> **Tip**: To build and test regular expressions, you can use RegEx tester tools such as **regex101**.
> This tool not only helps you create regular expressions, but also helps you **learn and debug** them effectively.

Using RegEx in Python

A **Regular Expression (RegEx)** is a special text string that helps to **find patterns in data.** A RegEx can be used to **check if a pattern exists** in a string, extract data, or replace patterns.

To use RegEx in Python, you first need to import the `re` module:

```
import re
```

Python RegEx Methods

To find a pattern in a string, we use different methods provided by the `re` module.

Common `re` methods:

- `re.findall(pattern, string)`
  Returns a **list of all non-overlapping matches** of the pattern in the string.
- `re.split(pattern, string)`
  Splits the string at each match of the pattern and returns a **list of substrings**.
- `re.sub(pattern, repl, string)`
  Replaces **one or many matches** of the pattern in the string with `repl`.
- `re.search(pattern, string)`
  Returns a **match object** if the pattern is found **anywhere** in the string (including multiline).
- `re.match(pattern, string)`
  Searches **only at the beginning** of the string. Returns a match object if found, else returns `None`.

Note: To use these methods, first import the module:

```
import re
```

1. `re.findall()`

The `re.findall()` method returns a **list of all non-overlapping matches** of a pattern in a string.
If the pattern is not found, it returns an **empty list**.

Example 1: Extract numbers from an Indian context

In [26]:
```
import re

string = 'Delhi 110001 Mumbai 400001 Kolkata 700001'
pattern = r'\d+'  # \d+ matches one or more digits

result = re.findall(pattern, string)
print(result)
```

```
['110001', '400001', '700001']
```

Explanation:

- `\d+` matches sequences of digits.
- `re.findall()` returns **all occurrences** in a list.
- If no match is found, the list will be **empty (`[]`)**.
- In this example, it extracts **Indian postal PIN codes** from the string.

Example 2: Case-insensitive search with `re.findall()`

In [27]:
```
import re

txt = '''Python Language is widely taught in IITs and NITs across India.
Many students in India start learning python as their first programming language.''

# Find all occurrences of the word 'language' (case-insensitive)
matches = re.findall('language', txt, re.I)
print(matches)
```

```
['Language', 'language']
```

Explanation:

- The pattern `'language'` matches the exact substring `"language"`.
- The flag `re.I` makes the search **case-insensitive**, so it matches both `"Language"` and `"language"`.
- `re.findall()` returns **all non-overlapping matches** as a **list**.
- In this example, `"language"` appears **twice** in the string:
  1. `"Python Language is widely taught..."` → matches `"Language"`
  2. `"programming language"` → matches `"language"`
- Hence, the result is `['Language', 'language']`.

Example 3: Case-insensitive search for 'python' with `re.findall()`

In [28]:
```
import re

txt = '''Python is the most beautiful language that a human being has ever created.
I recommend python for a first programming language'''

# Find all occurrences of 'python' (case-insensitive)
matches = re.findall('python', txt, re.I)
print(matches)
```

```
['Python', 'python']
```

Explanation:

- The pattern `'python'` matches the exact substring `"python"`.

- The flag `re.I` makes the search **case-insensitive**, so it matches both `"Python"` and `"python"`.
- `re.findall()` returns **all non-overlapping matches** as a **list**.
- In this example, the word `"python"` appears **twice** in the string, regardless of case.
- If the `re.I` flag were **not used**, only the exact case `"python"` would be matched.

Example 4: Using alternation and character sets with `re.findall()`

In [29]:
```
import re

txt = '''Python is the most beautiful language that a human being has ever created.
I recommend python for a first programming language'''

# Using alternation (|) to match 'Python' or 'python'
matches = re.findall('Python|python', txt)
print(matches)  # ['Python', 'python']
```

```
['Python', 'python']
```

In [30]:
```
# Using character set [Pp] to match 'Python' or 'python'
matches = re.findall('[Pp]ython', txt)
print(matches)  # ['Python', 'python']
```

```
['Python', 'python']
```

Explanation:

- `'Python|python'` uses the **alternation operator** `|` to match either `"Python"` or `"python"`.
- `'[Pp]ython'` uses a **character set** `[Pp]` to match either uppercase `"P"` or lowercase `"p"` at the start, followed by `"ython"`.
- Both methods return **all non-overlapping matches** in the string as a **list**: `['Python', 'python']`.
- This approach can be used **without the** `re.I` **flag** to handle case variations explicitly.

2. `re.split()`

The `re.split()` method **splits a string at each match of the pattern** and returns a **list of substrings**.

- It is useful when you want to break a string into parts using a **regex pattern** instead of a fixed character.
- If the pattern is not found, the returned list contains the **original string** as a single element.

Example 1: Splitting text into lines with re.split()

In [1]:
```python
import re

txt = '''IITs and NITs are the top engineering colleges in India.
Many students prepare for JEE to get admission in IITs.
Studying in IITs opens opportunities in research and global companies.
Does this inspire you to prepare for IIT?'''

# Split the text using \n (newline character)
result = re.split('\n', txt)
print(result)
```

```
['IITs and NITs are the top engineering colleges in India.', 'Many students prepare
for JEE to get admission in IITs.', 'Studying in IITs opens opportunities in researc
h and global companies.', 'Does this inspire you to prepare for IIT?']
```

Explanation:

- The pattern `\n` represents the newline character.
- `re.split('\n', txt)` splits the string at every newline.
- The result is a list of sentences, where each line becomes a separate element.
- In this example, the paragraph is divided into four sentences based on line breaks.

Example 2: Splitting text at numbers with re.split()

In [2]:
```python
import re

string = 'Delhi110092Mumbai400001'
pattern = r'\d+'  # Matches one or more digits

result = re.split(pattern, string)
print(result)
```

```
['Delhi', 'Mumbai', '']
```

Explanation:

- The pattern `\d+` matches one or more digits.
- `re.split('\d+', string)` splits the string wherever digits occur.
- The digits (`110092` , `400001`) are removed from the result.
- The output is a list of city names, while the numbers act as split points.

Explanation:

- If the pattern is **not found**, `re.split` returns a list containing the **original string**.
- You can pass the `maxsplit` argument to the `re.split` method.
- `maxsplit` defines the **maximum number of splits** that will occur.

Example 3: Splitting with a limit using re.split() and maxsplit

In [4]:
```python
import re

string = 'Kolkata700001Delhi110092Mumbai400001'
pattern = r'\d+'  # Matches one or more digits

# maxsplit = 1 → split only at the first occurrence
result = re.split(pattern, string, 1)
print(result)
```

```
['Kolkata', 'Delhi110092Mumbai400001']
C:\Users\goura\AppData\Local\Temp\ipykernel_25564\2037244303.py:7: DeprecationWarnin
g: 'maxsplit' is passed as positional argument
  result = re.split(pattern, string, 1)
```

Explanation:

- The pattern `\d+` matches one or more digits.
- By default, `re.split()` splits at **all matches**, but here we used `maxsplit=1`.
- This means the string is split only at the **first occurrence** of digits.
- The result keeps the first split point (`700001`) while leaving the rest untouched.

Note:

- The default value of `maxsplit` is **0**, which means **all possible splits** will be performed.

3. `re.sub()`

The `re.sub()` method **replaces all occurrences of a regex pattern** in a string with a specified replacement.

- It is useful when you want to **modify or clean text** by substituting matched parts.
- The method returns a **new string** with replacements applied (the original string remains unchanged).
- If the pattern is not found, the method simply returns the **original string** without changes.

Syntax:

```
re.sub(pattern, replace, string)
```

Example 1: Remove all whitespaces using `re.sub()`

In [5]:
```python
# Program to remove all whitespaces
import re

# multiline string
```

```
string = 'abc 12\
de 23 \n f45 6'

# matches all whitespace characters
pattern = '\s+'

# empty string
replace = ''

new_string = re.sub(pattern, replace, string)
print(new_string)
# Output: abc12de23f456
```

```
abc12de23f456
```

Explanation:

- The pattern `\s+` matches **one or more whitespace characters** (spaces, tabs, newlines).
- The replace value is an empty string (`''`), so all matched whitespaces are **removed.**
- `re.sub()` replaces all matches in the given string and returns a **new string.**
- In this example, the original string with spaces and newlines is transformed into: `abc12de23f456`

Explanation:

- If the pattern is **not found**, `re.sub()` returns the **original string.**
- You can pass `count` as the **fourth parameter** to the `re.sub()` method.
- If omitted, `count` defaults to **0**, which means **all occurrences** will be replaced.

Example 2: Using `count` with `re.sub()`

In [6]:
```
import re

# multiline string
string = 'abc 12\
de 23 \n f45 6'

# matches all whitespace characters
pattern = '\s+'
replace = ''

# replace only the first occurrence of whitespace
new_string = re.sub(r'\s+', replace, string, 1)
print(new_string)

# Output:
# abc12de 23
# f45 6
```

```
abc12de 23
 f45 6
```

```
C:\Users\goura\AppData\Local\Temp\ipykernel_25564\1824867832.py:12: DeprecationWarni
ng: 'count' is passed as positional argument
  new_string = re.sub(r'\s+', replace, string, 1)
```

Explanation:

- The pattern `\s+` matches **one or more whitespace characters** (spaces, tabs, or newlines).
- The parameter `count=1` ensures that only the **first match** is replaced.
- As a result, only the first whitespace (`' '`) after `abc` is removed, while the rest remain.
- If `count` were `0` (default), **all whitespaces** in the string would be removed.

Example 3: Replace all occurrences of 'Python' (case-insensitive) using `re.sub()`

In [7]:
```python
import re

txt = '''Python is the most beautiful language that a human being has ever created.
I recommend python for a first programming language'''

# Replace 'Python' or 'python' with 'JavaScript'
match_replaced = re.sub('Python|python', 'JavaScript', txt, re.I)
print(match_replaced)
# Output:
# JavaScript is the most beautiful language that a human being has ever created.
# I recommend JavaScript for a first programming language

# OR using character set
match_replaced = re.sub('[Pp]ython', 'JavaScript', txt, re.I)
print(match_replaced)
# Output:
# JavaScript is the most beautiful language that a human being has ever created.
# I recommend JavaScript for a first programming language
```

```
JavaScript is the most beautiful language that a human being has ever created.
I recommend JavaScript for a first programming language
JavaScript is the most beautiful language that a human being has ever created.
I recommend JavaScript for a first programming language
```

```
C:\Users\goura\AppData\Local\Temp\ipykernel_25564\4229837610.py:7: DeprecationWarnin
g: 'count' is passed as positional argument
  match_replaced = re.sub('Python|python', 'JavaScript', txt, re.I)
C:\Users\goura\AppData\Local\Temp\ipykernel_25564\4229837610.py:14: DeprecationWarni
ng: 'count' is passed as positional argument
  match_replaced = re.sub('[Pp]ython', 'JavaScript', txt, re.I)
```

Explanation:

- The pattern `'Python|python'` uses the **alternation operator (|)** to match either `"Python"` or `"python"`.
- The pattern `'[Pp]ython'` uses a **character set** `[Pp]` to match either uppercase `P` or lowercase `p` at the start of `"ython"`.
- The `re.I` flag makes the search **case-insensitive**, so it matches both `"Python"` and `"python"`.
- `re.sub()` replaces all **non-overlapping matches** with the string `"JavaScript"`.
- The result is a **new string** where every occurrence of `"Python"` / `"python"` is replaced.

Example 4: Remove all '%' characters using `re.sub()`

In [8]:
```
import re

txt = '''%I a%m te%%a%%che%r% a%n%d %% I l%o%ve te%ach%ing.
T%he%re i%s n%o%th%ing as r%ewarding a%s e%duc%at%i%ng a%n%d e%m%p%ow%er%ing p%e%o%
I fo%und te%a%ching m%ore i%n%t%er%%es%ting t%h%an any other %jobs.
D%o%es thi%s m%ot%iv%a%te %y%o%u to b%e a t%e%a%cher?'''

# Remove all '%' characters
matches = re.sub('%', '', txt)
print(matches)
```

```
I am teacher and  I love teaching.
There is nothing as rewarding as educating and empowering people.
I found teaching more interesting than any other jobs.
Does this motivate you to be a teacher?
```

4. `re.subn()`

The **`re.subn()`** method is similar to **`re.sub()`**, but instead of returning just the modified string, it returns a **tuple of two items**:

1. The new string with substitutions applied.
2. The **number of substitutions** made.

Syntax:

```
re.subn(pattern, replace, string, count=0)
```

Parameters of `re.sub()`:

- **pattern** → the regex pattern to search for
- **replace** → the string to replace matches with
- **string** → the input string
- **count** → *(optional)* maximum number of replacements; default is **0** (replace all)

Example 1: re.subn()

In [9]:
```python
# Program to remove all whitespaces
import re

# Multiline string
string = 'abc 12\
de 23 \n f45 6'

# Matches all whitespace characters
pattern = '\s+'

# Replace with empty string
replace = ''

new_string = re.subn(pattern, replace, string)
print(new_string)
# Output: ('abc12de23f456', 4)
```

```
('abc12de23f456', 4)
```

Explanation:

- `\s+` matches **one or more whitespace characters** (spaces, tabs, newlines).
- `re.subn()` replaces all matches with the empty string (`''`).
- It returns a **tuple**:
  1. The **modified string**: `'abc12de23f456'`
  2. The **number of substitutions** made: `4`

5. `re.search()`

The **`re.search()`** method takes two arguments: a pattern and a string. The method looks for the first location where the RegEx pattern produces a match with the string.

If the search is successful, **`re.search()`** returns a match object; if not, it returns **`None`**.

```
match = re.search(pattern, str)
```

Example 1: re.search()

In [10]:
```python
import re

string = "Python is fun"

# Check if 'Python' is at the beginning
match = re.search(r'\APython', string)

if match:
    print("pattern found inside the string")
else:
```

```
    print("pattern not found")

# Output: pattern found inside the string
```

```
pattern found inside the string
```

Explanation:

- `\A` matches the **start of the string**.
- `re.search()` scans the string and returns a **match object** if the pattern is found anywhere.
- If the pattern is **not found**, it returns `None`.
- In this example, the string starts with `"Python"`, so a **match is found**.

Example 2: re.search()

In [11]:
```
import re

txt = '''Python is the most beautiful language that a human being has ever created.
I recommend python for a first programming language'''

# Search for the word 'first' (case-insensitive)
match = re.search('first', txt, re.I)
print(match)  # <re.Match object; span=(100, 105), match='first'>

# Get the span (start and end positions) of the match
span = match.span()
print(span)  # (100, 105)

# Extract start and end positions
start, end = span
print(start, end)  # 100 105

# Extract the matched substring using slice
substring = txt[start:end]
print(substring)  # first
```

```
<re.Match object; span=(100, 105), match='first'>
(100, 105)
100 105
first
```

Explanation:

- `re.search()` returns a **match object** for the first occurrence of the pattern.
- `span()` gives a **tuple of start and end positions** of the match in the string.
- Using string slicing `txt[start:end]`, you can **extract the matched substring**.
- The flag `re.I` makes the search **case-insensitive**, so it matches `"First"`, `"first"`, or `"FIRST"`.

Match Object

In Python's `re` module, a **match object** is returned by functions like `re.search()` or `re.match()` when a pattern is found in a string.
It contains information about the match and allows you to extract details about the found substring.

Key Attributes and Methods of a Match Object

Attribute / Method	Description
`.group()`	Returns the **matched substring**. Equivalent to `string[start:end]`.
`.start()`	Returns the **starting index** of the match.
`.end()`	Returns the **ending index** of the match.
`.span()`	Returns a **tuple (start, end)** indicating the span of the match.
`.re`	Returns the **regular expression object** used for the match.
`.string`	Returns the **original string** that was searched.

Example:

In [12]:
```python
import re

txt = "Python is fun"
match = re.search("Python", txt)

if match:
    print("Matched substring:", match.group())  # Python
    print("Start index:", match.start())        # 0
    print("End index:", match.end())            # 6
    print("Span:", match.span())                # (0, 6)
    print("Original string:", match.string)     # Python is fun
```

```
Matched substring: Python
Start index: 0
End index: 6
Span: (0, 6)
Original string: Python is fun
```

Explanation:

- `match.group()` returns the **exact substring** that matched the pattern.
- `match.start()` and `match.end()` give the **positions** of the match in the string.
- `match.span()` combines **start and end** into a tuple.
- `match.string` gives the **original string** that was searched.

- Match objects are useful for obtaining **detailed information** about the location and content of the matched text.

Example: Using `re.match()` and Match Object

In [14]:
```python
import re

txt = 'I love to teach python and javaScript'

# re.match() checks for a match only at the beginning of the string
match = re.match('I love to teach', txt, re.I)
print(match)  # <re.Match object; span=(0, 15), match='I love to teach'>

# Get the start and end positions of the match
span = match.span()
print(span)     # (0, 15)

start, end = span
print(start, end)  # 0 15

# Extract the matched substring
substring = txt[start:end]
print(substring)  # I love to teach
```

```
<re.Match object; span=(0, 15), match='I love to teach'>
(0, 15)
0 15
I love to teach
```

Explanation:

- `re.match()` checks for a match **only at the beginning** of the string.
- The returned **match object** contains information about the match.
- `match.span()` returns the **start and end indices** of the match as a tuple `(start, end)`.
- `txt[start:end]` extracts the **matched substring** using the span.
- This is useful when you want both the **value and position** of the matched pattern in the string.

Example: `re.match()` with No Match

In [15]:
```python
import re

txt = 'I love to teach python and javaScript'

# Attempt to match a different pattern at the beginning
match = re.match('I like to teach', txt, re.I)
print(match)  # None
```

```
None
```

Explanation:

- `re.match()` only checks for a match **at the beginning** of the string.
- Here, the string starts with `"I love to teach"`, not `"I like to teach"`.
- Since the pattern does not match, `re.match()` returns `None`.
- This demonstrates that `re.match()` is **strict about the starting position.**

1. `match.group()`

The `group()` method of a match object returns the **exact substring** of the string where the pattern matched.

Example:

In [16]:
```
import re

txt = 'I love to teach python and javaScript'

match = re.match('I love to teach', txt, re.I)
print(match.group())  # I love to teach
```

```
I love to teach
```

Explanation:

- `match.group()` returns the **matched portion** of the string.
- In this example, the pattern `"I love to teach"` matches the start of `txt`.
- Therefore, `match.group()` returns `"I love to teach"`.
- If there is **no match**, calling `group()` will raise an `AttributeError`.

Example 6: Using Match Object with Groups

In [17]:
```
import re

string = '39801 356, 2102 1111'

# Pattern: three digits followed by a space followed by two digits
pattern = '(\d{3}) (\d{2})'

# Perform the search
match = re.search(pattern, string)

if match:
    print(match.group())  # 801 35
else:
    print("pattern not found")
```

```
801 35
```

Explanation:

- The pattern `(\d{3}) (\d{2})` uses **parentheses to define groups**:
  - `(\d{3})` matches **exactly three digits**
  - `(\d{2})` matches **exactly two digits**
- `match` is a **Match object** returned by `re.search()`.
- `match.group()` returns the **entire matched substring**, including both groups.
- In this example, the substring `"801 35"` matches the pattern and is returned.
- If the pattern is **not found**, `match` will be `None`.

Match Object Methods for Groups

When using **parentheses** `()` in a regex pattern, you can extract matched subgroups using several methods:

In [18]:
```python
import re

string = '39801 356, 2102 1111'
pattern = '(\d{3}) (\d{2})'

match = re.search(pattern, string)

if match:
    print("Full match:", match.group())       # Entire matched string
    print("Group 1:", match.group(1))         # First subgroup (\d{3})
    print("Group 2:", match.group(2))         # Second subgroup (\d{2})
    print("Groups (1,2):", match.group(1, 2))# Tuple of selected groups
    print("All groups:", match.groups())      # Tuple of all subgroups
else:
    print("pattern not found")
```

```
Full match: 801 35
Group 1: 801
Group 2: 35
Groups (1,2): ('801', '35')
All groups: ('801', '35')
```

Explanation:

- `match.group()` → Returns the **entire match**.
- `match.group(1)` → Returns the **first parenthesized subgroup**.
- `match.group(2)` → Returns the **second parenthesized subgroup**.
- `match.group(1, 2)` → Returns a **tuple containing the specified groups** `(group1, group2)`.
- `match.groups()` → Returns a **tuple of all subgroups** in the pattern.
- Using groups helps **extract specific parts** of the matched string efficiently.

2. `match.start()`, `match.end()` and `match.span()`

The **`start()`** and **`end()`** methods of a match object help locate the positions of matches in the original string.

- `match.start()` → Returns the **starting index** of the match.
- `match.end()` → Returns the **ending index** of the match (exclusive).
- `match.span()` → Returns a **tuple** `(start, end)` representing the start and end indices of the match.

In [19]:
```python
import re

txt = 'I love to teach python and javaScript'

# Match a substring at the beginning
match = re.match('I love to teach', txt)

if match:
    print("Start index:", match.start())   # 0
    print("End index:", match.end())       # 15
    print("Span:", match.span())           # (0, 15)
    print("Matched substring:", txt[match.start():match.end()])  # 'I love to teach
else:
    print("pattern not found")
```

```
Start index: 0
End index: 15
Span: (0, 15)
Matched substring: I love to teach
```

Explanation:

- `start()` and `end()` are useful to know **where a match occurs** in the string.
- `span()` provides both **start and end indices together**.
- You can use these indices to **extract or manipulate** the matched portion of the string.

3. `match.re` and `match.string`

The **`re`** and **`string`** attributes of a match object provide information about the original regular expression and the input string.

- `match.re` → Returns the **compiled regular expression object** used for matching.
- `match.string` → Returns the **original string** that was searched.

In [20]:
```python
import re

txt = 'I love to teach python and javaScript'

# Match a substring
```

```python
match = re.search('teach', txt)

if match:
    print("Regular Expression object:", match.re)   # <_sre.SRE_Pattern object; ...
    print("Original string:", match.string)        # 'I love to teach python and ja
    print("Matched substring:", match.group())     # 'teach'
else:
    print("pattern not found")
```

```
Regular Expression object: re.compile('teach')
Original string: I love to teach python and javaScript
Matched substring: teach
```

Explanation:

- `match.re` lets you access the **pattern object** that was used for the match.
- `match.string` allows you to see the **exact string** that was searched.
- These attributes are useful for **debugging or reusing** the regex object.

Using `r` Prefix Before RegEx

In Python, prefixing a string with `r` or `R` creates a **raw string**, which treats backslashes (`\`) as literal characters and not as escape characters.

- Example:
  - `'\n'` → Interpreted as a **newline.**
  - `r'\n'` → Interpreted as two characters: a **backslash** `\` followed by `n`.

Why use raw strings in RegEx?

- In regular expressions, the backslash (`\`) is used to escape special characters (e.g., `\d`, `\w`, `\s`).
- Without the `r` prefix, Python might interpret `\` as an escape character, which can lead to errors.
- Using `r'...'` ensures the backslash is **treated literally**, making the regex easier to read and write.

In [21]:
```python
import re

# Without raw string
pattern = '\\d+'   # Need double backslash to match digits

# With raw string
pattern_raw = r'\d+'  # Single backslash is enough

txt = 'There are 123 apples and 45 oranges.'

print(re.findall(pattern, txt))       # ['123', '45']
print(re.findall(pattern_raw, txt))   # ['123', '45']
```

```
['123', '45']
['123', '45']
```

Explanation:

- Both `\\d+` and `r'\d+'` match **one or more digits**.
- Using the `r` **prefix** (raw string) simplifies regex writing and avoids confusion with **Python escape sequences**.

Example 7: Raw String Using `r` Prefix

In [22]:
```
import re

string = '\n and \r are escape sequences.'

# Match newline (\n) and carriage return (\r) characters
result = re.findall(r'[\n\r]', string)
print(result)
# Output: ['\n', '\r']
```

```
['\n', '\r']
```

Explanation:

- `r'[\n\r]'` is a **raw string**, so backslashes are treated literally.
- The character set `[\n\r]` matches any **newline (`\n`)** or **carriage return (`\r`)** in the string.
- `re.findall()` returns **all non-overlapping matches** as a list.
- In this example, the string contains one newline (`\n`) and one carriage return (`\r`), so the output is `['\n', '\r']`.

Example of RegEx with Metacharacters

Let us use examples to clarify the **metacharacters** with RegEx methods.

Square Brackets `[]`

Square brackets `[]` are used to specify a **set of characters** to match.
They allow matching **any one character** from the set.

Example: Match vowels (both lowercase and uppercase)

In [23]:
```
import re

txt = "India is a beautiful country."
```

```
# Match all vowels (a, e, i, o, u) in both lowercase and uppercase
pattern = r'[aeiouAEIOU]'
matches = re.findall(pattern, txt)
print(matches)
# Output: ['I', 'i', 'a', 'i', 'a', 'e', 'u', 'i', 'o', 'u']
```

```
['I', 'i', 'a', 'i', 'a', 'e', 'a', 'u', 'i', 'u', 'o', 'u']
```

Explanation:

- `[aeiouAEIOU]` matches **any single character** in the set (lowercase or uppercase vowels).
- `re.findall()` returns **all non-overlapping matches** as a list.
- In this example, all **vowels** in the string are captured.

Example : Using Square Brackets `[]` to Handle Case

In [24]:
```
import re

regex_pattern = r'[Aa]pple'  # Matches "Apple" or "apple"
txt = 'Apple and banana are fruits. An old cliche says an apple a day a doctor way

matches = re.findall(regex_pattern, txt)
print(matches)  # ['Apple', 'apple']
```

```
['Apple', 'apple']
```

Explanation:

- `[Aa]` matches **either uppercase A or lowercase a** at the start of the word.
- `pple` matches the **literal substring** `"pple"` following the first character.
- `re.findall()` returns **all non-overlapping matches** as a list.
- In this example, it finds both `"Apple"` and `"apple"` in the string.

Example 2: Using Square Brackets `[]` with Alternation `|`

In [25]:
```
import re

regex_pattern = r'[Aa]pple|[Bb]anana'  # Matches "Apple"/"apple" or "Banana"/"banan
txt = 'Apple and banana are fruits. An old cliche says an apple a day a doctor way

matches = re.findall(regex_pattern, txt)
print(matches)  # ['Apple', 'banana', 'apple', 'banana']
```

```
['Apple', 'banana', 'apple', 'banana']
```

Explanation:

- `[Aa]pple` matches `"Apple"` or `"apple"`.

- `[Bb]anana` matches `"Banana"` or `"banana"`.
- The **alternation operator** `|` allows matching **either** of the patterns.
- `re.findall()` returns **all non-overlapping matches** in the string.
- In this example, it finds `"Apple"`, `"banana"`, `"apple"`, and `"banana"`.

Escape Character `\`

In Python RegEx, the backslash `\` is used to **escape special characters** so that they are treated literally.

- For example, `$` normally represents the **end of a string**, but `\$` matches the **literal** `$` character.
- Similarly, `.` normally matches **any character**, but `\.` matches a literal dot.
- Using a **raw string** with the `r` prefix (e.g., `r"\$"`) ensures that Python treats `\` as a literal backslash and not as an escape sequence.

Example:

In [26]:
```
import re

txt = "The price is $100."

# Match the literal dollar sign
matches = re.findall(r'\$', txt)
print(matches)  # ['$']
```

```
['$']
```

Explanation:

- `\$` matches the **literal** `$` in the string.
- `re.findall()` returns **all occurrences** of the escaped pattern.
- Using `r` before the string prevents Python from interpreting `\` as an escape character.

Example : Using `\d` to match digits

In [27]:
```
import re

regex_pattern = r'\d'  # \d matches any single digit
txt = "Hawking born on 8 January 1942 and died on 14 March 2018. Einstein's birth a

matches = re.findall(regex_pattern, txt)
print(matches)  # ['8', '1', '9', '4', '2', '1', '4', '2', '0', '1', '8', '7', '6']
```

```
['8', '1', '9', '4', '2', '1', '4', '2', '0', '1', '8', '7', '6']
```

Explanation:

- `\d` matches **every single digit** in the string.
- `re.findall()` returns **all non-overlapping matches** as a list.
- Here, each **individual digit** is returned separately, not complete numbers.
- This approach is **not ideal** if we want full numbers like `1942`, `2018`, or `76`.

One or More Occurrences `+`

The `+` quantifier matches **one or more occurrences** of the preceding pattern.

Example: Extract full numbers using `\d+`

In [28]:
```
import re

regex_pattern = r'\d+'  # \d+ matches one or more digits consecutively
txt = "Hawking born on 8 January 1942 and died on 14 March 2018. Einstein's birth a

matches = re.findall(regex_pattern, txt)
print(matches)  # ['8', '1942', '14', '2018', '76']
```

```
['8', '1942', '14', '2018', '76']
```

Explanation:

- `\d+` matches **sequences of one or more digits** together.
- `re.findall()` returns **all non-overlapping matches** as a list.
- Unlike `\d`, which returns each digit separately, `\d+` returns **whole numbers.**
- In this example, numbers like `1942` and `2018` are captured as **complete numbers.**

Example : One or more occurrences with `+`

In [29]:
```
import re

regex_pattern = r'\d+'  # \d+ matches one or more consecutive digits
txt = "Hawking born on 8 January 1942 and died on 14 March 2018 Einstein's birth an

matches = re.findall(regex_pattern, txt)
print(matches)  # ['8', '1942', '14', '2018', '76']
```

```
['8', '1942', '14', '2018', '76']
```

Explanation:

- `\d+` matches **one or more consecutive digits.**
- `re.findall()` returns **all non-overlapping matches** as a list.
- Unlike `\d`, which matches each digit separately, `\d+` returns **whole numbers.**
- In this example, numbers like `1942` and `2018` are captured as **complete numbers.**

Example: Period `.`

The period `.` in regex matches **any single character except a newline** (`\n`).

In [30]:
```
import re

regex_pattern = r'J.n'  # Matches 'J' followed by any character and then 'n'
txt = "Hawking born on 8 January 1942 and died on 14 March 2018"

matches = re.findall(regex_pattern, txt)
print(matches)  # ['Jan']
```

```
['Jan']
```

Explanation:

- `.` matches **any single character except newline.**
- `J.n` matches a substring starting with `'J'`, followed by **any character**, and ending with `'n'`.
- In the example, `'Jan'` in `'January'` is matched.

Example: Character class with period `.`

In [31]:
```
import re

regex_pattern = r'[a].'  # [a] matches 'a', . matches any character except newline
txt = 'Apple and Banana are fruits'

matches = re.findall(regex_pattern, txt)
print(matches)  # ['an', 'an', 'an', 'a ', 'ar']
```

```
['an', 'an', 'an', 'a ', 'ar']
```

Explanation:

- `[a]` matches the **character 'a'**.
- `.` matches **any single character except a newline.**
- Together, `[a].` matches `'a'` followed by **any character.**
- In the example, it finds all occurrences of `'a'` followed by the next character: `'an'`, `'an'`, `'an'`, `'a '`, `'ar'`.

Example: Character class with `+`

In [32]:
```
import re

txt = 'Apple and Banana are fruits'
regex_pattern = r'[a].+'  # [a] matches 'a', .+ matches one or more of any characte
```

```
matches = re.findall(regex_pattern, txt)
print(matches)  # ['and Banana are fruits']
```

```
['and Banana are fruits']
```

Explanation:

- `[a]` matches the **character 'a'**.
- `.+` matches **one or more of any character** except newline.
- Together, `[a].+` matches `'a'` followed by **everything until the end of the string** (because `+` is greedy).
- In this example, it returns `'and Banana are fruits'` since it starts from the first `'a'` and continues to the end.

Zero or more times `*`

In [33]:
```
import re

txt = 'Apple and Banana are fruits'
regex_pattern = r'[a].*'  # [a] matches 'a', .* matches zero or more of any charact

matches = re.findall(regex_pattern, txt)
print(matches)  # ['and Banana are fruits']
```

```
['and Banana are fruits']
```

Explanation:

- `[a]` matches the **character 'a'**.
- `.*` matches **zero or more of any character** except newline.
- Together, `[a].*` matches `'a'` followed by **everything until the end of the string** (greedy match).
- In this example, it returns `'and Banana are fruits'` because it starts from the first `'a'` and includes everything after.

Zero or one time `?`

In [34]:
```
import re

txt = 'Apple and Banana are fruits'
regex_pattern = r'Ba?na'  # 'a?' matches zero or one occurrence of 'a'

matches = re.findall(regex_pattern, txt)
print(matches)  # ['Bana', 'Bana']
```

```
['Bana']
```

Explanation:

- `a?` matches **zero or one occurrence** of `'a'`.
- `'Ba?na'` matches both `'Bana'` (with one `'a'`) and would also match `'Bna'` if it existed.
- `re.findall()` returns **all non-overlapping matches** in a list.
- In this example, `'Bana'` appears twice in the string, so the output is `['Bana', 'Bana']`.

Example : Zero or one occurrence of a character `?`

In [35]:
```
import re

txt = '''I am not sure if there is a convention how to write the word e-mail.
Some people write it as email others may write it as Email or E-mail.'''

regex_pattern = r'[Ee]-?mail'  # '-?' means the hyphen is optional

matches = re.findall(regex_pattern, txt)
print(matches)  # ['e-mail', 'email', 'Email', 'E-mail']
```

```
['e-mail', 'email', 'Email', 'E-mail']
```

Explanation:

- `-?` matches **zero or one occurrence** of the hyphen.
- `[Ee]` matches either **uppercase 'E'** or **lowercase 'e'**.
- `[Ee]-?mail` matches `"email"`, `"e-mail"`, `"Email"`, and `"E-mail"`.
- `re.findall()` returns **all non-overlapping matches** as a list.
- This allows **flexibility** to match words **with or without a hyphen**.

Quantifier `{}`

Curly brackets `{}` allow you to specify **exactly how many times** a pattern should occur.

- `{n}` → exactly **n** occurrences
- `{n,}` → at least **n** occurrences
- `{n,m}` → between **n** and **m** occurrences

Example : Match exactly 4 digits

In [36]:
```
import re

txt = "My PIN codes are 1100, 22011, 330, and 4400."
regex_pattern = r'\d{4}'  # exactly 4 digits
```

```
matches = re.findall(regex_pattern, txt)
print(matches)  # ['1100', '4400']
```

```
['1100', '2201', '4400']
```

Explanation:

- `\d{4}` matches **exactly four digits**.
- `re.findall()` returns **all non-overlapping matches** in a list.
- Strings with **fewer or more than 4 digits** are ignored.

Example : Match exactly 4 digits

In [37]:
```
import re

txt = "Hawking born on 8 January 1942 and died on 14 March 2018 Einstein's birth an
regex_pattern = r'\d{4}'  # exactly four digits

matches = re.findall(regex_pattern, txt)
print(matches)  # Output: ['1942', '2018']
```

```
['1942', '2018']
```

Explanation:

- `\d{4}` matches **exactly four digits**.
- `re.findall()` returns **all non-overlapping matches** in a list.
- Strings with **fewer or more than 4 digits** are ignored.

Example : Match 1 to 4 digits

In [38]:
```
import re

txt = "Hawking born on 8 January 1942 and died on 14 March 2018 Einstein's birth an
regex_pattern = r'\d{1,4}'   # matches 1 to 4 digits

matches = re.findall(regex_pattern, txt)
print(matches)  # Output: ['8', '1942', '76', '14', '2018', '76']
```

```
['8', '1942', '14', '2018', '76']
```

Explanation:

- The pattern `\d{1,4}` matches **numbers with 1 to 4 digits.**
- `re.findall()` returns **all non-overlapping matches** as a list.
- In this example, numbers like `8`, `14`, `76`, `1942`, and `2018` are matched because they have **between 1 and 4 digits.**
- Each occurrence is captured **individually**, preserving the order in the text.

Caret `^` - Start of String

- The caret `^` is a **metacharacter** in regular expressions that matches the **start of a string**.
- If a pattern starts with `^`, it will **only match** if the string **begins with the pattern**.
- It is often used to ensure that a string **starts with a certain substring**.

In [39]:
```python
import re

txt = "Python is fun"
pattern = r'^Python'  # ^ matches start of string

match = re.search(pattern, txt)
if match:
    print("Pattern found at the start of the string")
else:
    print("Pattern not found")
```

```
Pattern found at the start of the string
```

Explanation:

- `^Python` looks for the substring `"Python"` **at the very beginning** of `txt`.
- If `"Python"` appeared **later** in the string, the match would **not occur**.
- `re.search()` returns a **match object** if found, otherwise `None`.

In [41]:
```python
import re

txt = "Hawking born on 8 January 1942 and died on 14 March 2018 Einstein's birth an
regex_pattern = r'^Hawking'  # ^ matches the start of the string

matches = re.findall(regex_pattern, txt)
print(matches)  # Output: ['Hawking']
```

```
['Hawking']
```

Explanation:

- The caret `^` checks if the string **starts with the specified pattern**.
- Here, `^Hawking` matches `"Hawking"` **only if it appears at the beginning** of `txt`.
- `re.findall()` returns a **list of all matches**. Since `"Hawking"` is at the start, it returns `['Hawking']`.
- If the string started with any other word, the result would be an **empty list** `[]`.

Example: Negation `[^...]`

In [42]:
```python
import re

txt = "Hawking born on 8 January 1942 and died on 14 March 2018 Einstein's birth an
regex_pattern = r'[^A-Za-z ]+'  # ^ inside [] negates the set

matches = re.findall(regex_pattern, txt)
print(matches)  # Output: ['8', '1942', '14', '2018', "'", '(', '-', ')', '76']
```

```
['8', '1942', '14', '2018', "'", '(', '-', ')', '76']
```

Explanation:

- When `^` is used **inside square brackets** `[]`, it acts as a **negation operator**.
- `[^A-Za-z ]+` matches **one or more characters** that are **not uppercase A-Z, lowercase a-z, or a space**.
- `re.findall()` returns **all non-overlapping matches**.
- In this example, all **numbers, punctuation, and symbols** are matched because they are **not letters or spaces**.

Question : Most Frequent Word

Find the most frequent word in the following paragraph:

In [44]:
```python
paragraph = """I love teaching. If you do not love teaching what else can you love.
            I love Python if you do not love something which can give you all the c
```

In [45]:
```python
# Use re.findall() to extract all words from the paragraph.
words = re.findall(r'\b\w+\b', paragraph.lower())

# Use collections.Counter to count word frequencies.

from collections import Counter
freq = Counter(words)
most_common = freq.most_common()
print(most_common)
```

```
[('love', 6), ('you', 5), ('can', 3), ('i', 2), ('teaching', 2), ('if', 2), ('do',
2), ('not', 2), ('what', 2), ('else', 2), ('python', 1), ('something', 1), ('which',
1), ('give', 1), ('all', 1), ('the', 1), ('capabilities', 1), ('to', 1), ('develop',
1), ('an', 1), ('application', 1)]
```

Steps & Reasoning for Word Frequency Count:

1. **Tokenize the text** – split the paragraph into individual words.
2. **Ignore punctuation** and treat words separately.
3. **Count occurrences** of each word using a dictionary or `Counter`.
   - Example: `"love"` appears 6 times, `"you"` appears 5 times, etc.
4. **Sort the words by frequency** in descending order to get the most frequent words first.

Explanation: Word Frequency Using Regex

1. **Extract words with `re.findall()`**

- Pattern used: `r'\b\w+\b'`
  - `\b` → Word boundary, ensures we match **whole words.**
  - `\w+` → Matches **one or more alphanumeric characters** (letters, digits, underscore).
- `paragraph.lower()` → Converts all words to **lowercase** for uniform counting.
- Output: A **list of all words** in the paragraph.

2. **Count word frequencies with `Counter`**

- `Counter(words)` → Creates a **dictionary-like object** where **keys = words** and **values = counts.**
- `most_common()` → Returns a **list of tuples** `(word, frequency)` sorted in **descending order** of frequency.

Example:
If `"love"` appears 6 times and `"you"` 5 times, output will include:

```
[('love', 6), ('you', 5), ...]
```

Question 2: Extract Particle Positions and Find Distance

Problem:

The positions of particles on the horizontal x-axis are described in text:
`-12, -4, -3, -1 in the negative direction, 0 at origin, and 4, 8 in the positive direction.`

We need to:

1. **Extract all numbers** (particle positions) from the text.
2. **Find the distance** between the two furthest particles.

In [47]:
```python
import re

# Text with particle positions
text = """The position of some particles on the horizontal x-axis are -12, -4, -3 a
                0 at origin, 4 and 8 in the positive direction."""

# Extract numbers using regex
points = re.findall(r'-?\d+', text)  # -? allows negative numbers
print("Extracted points:", points)

# Convert strings to integers
points = [int(p) for p in points]
```

```
print("Integer points:", points)

# Find min and max positions
min_pos = min(points)
max_pos = max(points)

# Calculate distance between furthest particles
distance = max_pos - min_pos
print("Distance between furthest particles:", distance)
```

```
Extracted points: ['-12', '-4', '-3', '-1', '0', '4', '8']
Integer points: [-12, -4, -3, -1, 0, 4, 8]
Distance between furthest particles: 20
```

Explanation:

Regex Pattern: `-?\d+`

- `-?` → Matches an **optional negative sign**.
- `\d+` → Matches **one or more digits**.
- Together, `-?\d+` matches **both negative and positive integers.**

Finding Distance:

- `max(positions)` → Furthest particle in the **positive direction**.
- `min(positions)` → Furthest particle in the **negative direction.**
- `distance = max - min` → Gives the **distance between the two furthest particles**.

Result: The distance between the furthest particles is **20 units**.

Question m: Check if a String is a Valid Python Variable

Problem:

We want to write a **regex pattern** to check if a string is a **valid Python variable**.

Rules for Python variable names:

1. Must **start with a letter** (`a-z` , `A-Z`) or **underscore** (`_`).
2. Can contain **letters, digits** (`0-9`), and **underscores** after the first character.
3. **Cannot start with a digit**.

In [49]:
```
import re

def is_valid_variable(name):
    # Regex pattern:
    # ^       → start of string
    # [a-zA-Z_] → first character must be letter or underscore
    # [a-zA-Z0-9_]* → zero or more letters, digits, underscores
```

```
    # $       → end of string
    pattern = r'^[a-zA-Z_][a-zA-Z0-9_]*$'
    return bool(re.match(pattern, name))

# Test examples
print(is_valid_variable('first_name'))   # True
print(is_valid_variable('first-name'))   # False
print(is_valid_variable('1first_name'))  # False
print(is_valid_variable('firstname'))    # True
```

```
True
False
False
True
```

Explanation:

- `^[a-zA-Z_]` → Ensures the variable **starts with a letter or underscore.**
- `[a-zA-Z0-9_]*` → Allows **letters, digits, or underscores** after the first character.
- `$` → Ensures the pattern matches the **entire string.**
- `re.match()` → Checks the pattern **from the beginning** of the string.
- `bool()` → Converts the **match object** to `True` (valid) or `False` (invalid).

Question : Clean Text and Count Most Frequent Words

Problem:

We have a text containing **special characters**. Our tasks are:

1. **Clean the text** – remove all special characters.
2. **Count the three most frequent words** in the cleaned text.

In [50]:
```
import re
from collections import Counter

# Original messy text
sentence = '''%I $am@% a %tea@cher%, &and& I lo%#ve %tea@ching%;.
There $is nothing; &as& mo@re rewarding as educa@ting &and& @emp%o@wering peo@ple.
;I found tea@ching m%o@re interesting tha@n any other %jo@bs.
%Do@es thi%s mo@tivate yo@u to be a tea@cher!?'''

# Step 1: Clean the text by removing special characters
cleaned_text = re.sub(r'[^A-Za-z0-9\s]', '', sentence)
print(cleaned_text)
# Output: I am a teacher and I love teaching There is nothing as more rewarding as
#                                                    I found teaching more intere

# Step 2: Count the most frequent words
words = cleaned_text.split()
word_counts = Counter(words)
```

```python
most_common_three = word_counts.most_common(3)
print(most_common_three)
# Output: [(3, 'I'), (2, 'teaching'), (2, 'teacher')]
```

```
I am a teacher and I love teaching
There is nothing as more rewarding as educating and empowering people
I found teaching more interesting than any other jobs
Does this motivate you to be a teacher
[('I', 3), ('a', 2), ('teacher', 2)]
```

Explanation:

- `re.sub(r'[^A-Za-z0-9\s]', '', sentence)`
  - Removes everything **except letters, numbers, and spaces**.
  - All special characters like `%`, `@`, `#`, `;` are removed.
- `split()` → Converts the cleaned string into a **list of words**.
- `Counter(words)` → Counts **how many times each word appears.**
- `most_common(3)` → Returns the **three most frequent words.**

Regex Flags (Modifiers)

Regex flags change how patterns are interpreted.

They are passed as the third argument in regex functions, for example:

```python
re.findall(pattern, string, flags)
```

Flag	Name	Description
`re.I` or `re.IGNORECASE`	Ignore case	Matches letters regardless of case
`re.M` or `re.MULTILINE`	Multiline	`^` and `$` match the start and end of **each line**, not just the string
`re.S` or `re.DOTALL`	Dotall	`.` matches **newlines** as well
`re.X` or `re.VERBOSE`	Verbose	Allows whitespace and comments inside patterns for readability

In [ ]: